Priority queue notes

Step notes: `S&Q Applications/overall notes.md`

Tokenised: `expr` is a list. `for token in expr`

Postfix to infix

```
stack = Stack()

for token in expr:

    if token not in PRECEDENCE:
        stack.push(token)

    else:
        op2 = stack.pop()
        op1 = stack.pop()
        stack.push(f"({op1} {token} {op2})")

return stack.pop()
```

Postfix to prefix

Same `if` / `else`. This line changes:

```
stack.push(f"{token} {op1} {op2}")
```

Prefix to infix

```
stack = Stack()

for token in reversed(expr):

    if token not in PRECEDENCE:
        stack.push(token)

    else:
        op1 = stack.pop()
        op2 = stack.pop()
        stack.push(f"({op1} {token} {op2})")

return stack.pop()
```

Prefix to postfix

Same `if` / `else`. This line changes:

```
stack.push(f"{op1} {op2} {token}")
```

Evaluate postfix

```
stack = Stack()

for token in expr:

    if token.isdigit():
        stack.push(int(token))

    else:
        op2 = stack.pop()
        op1 = stack.pop()

        if token == "+":
            stack.push(op1 + op2)
        elif token == "-":
            stack.push(op1 - op2)
        elif token == "*":
            stack.push(op1 * op2)
        elif token == "/":
            stack.push(op1 / op2)
        elif token == "**":
            stack.push(op1 ** op2)

return stack.pop()
```

Evaluate prefix

```
stack = Stack()

for token in reversed(expr):

    if token.isdigit():
        stack.push(int(token))

    else:
        op1 = stack.pop()
        op2 = stack.pop()

        if token == "+":
            stack.push(op1 + op2)
```

```
        elif token == "-":
            stack.push(op1 - op2)
        elif token == "*":
            stack.push(op1 * op2)
        elif token == "/":
            stack.push(op1 / op2)
        elif token == "**":
            stack.push(op1 ** op2)

return stack.pop()
```

Infix to postfix

```
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/": 2, "**": 3}

operators = Stack()
output = Stack()

for token in expr:

    if token == "(":
        operators.push(token)

    elif token == ")":
        while not operators.isEmpty() and operators.peek() != "(":
            output.push(operators.pop())
        operators.pop()

    elif token.isalnum():
        output.push(token)

    else:
        while not operators.isEmpty() and operators.peek() != "(" and
(PRECEDENCE[operators.peek()] > PRECEDENCE[token] or (PRECEDENCE[operators.peek()]
== PRECEDENCE[token] and token != "**")):
            output.push(operators.pop())
        operators.push(token)

while not operators.isEmpty():
    output.push(operators.pop())

return output
```

Infix to prefix

```
operators = Stack()
output = Stack()

for token in reversed(expr):
```

```
    if token == ")":
        operators.push(token)

    elif token == "(":
        while not operators.isEmpty() and operators.peek() != ")":
            output.push(operators.pop())
        operators.pop()

    elif token.isalnum():
        output.push(token)

    else:
        while not operators.isEmpty() and operators.peek() != ")" and
(PRECEDENCE[operators.peek()] > PRECEDENCE[token] or (PRECEDENCE[operators.peek()]
== PRECEDENCE[token] and token == "**")):
            output.push(operators.pop())
        operators.push(token)

while not operators.isEmpty():
    output.push(operators.pop())

return output
```

Raw

One string. `for char in expression`

Postfix to infix — `postfix_to_infix.py`

```
stack = Stack()

for char in expression:

    if char.isspace():
        continue

    if char not in PRECEDENCE:
        stack.push(char)

    else:
        op2 = stack.pop()
        op1 = stack.pop()
        stack.push(f"({op1} {char} {op2})")

return stack.pop()
```

Postfix to prefix changes the push to `f"{char} {op1} {op2}"`.

Prefix to infix — `prefix_to_infix.py`

```
stack = Stack()

for char in reversed(expression):

    if char.isspace():
        continue

    if char not in PRECEDENCE:
        stack.push(char)

    else:
        op1 = stack.pop()
        op2 = stack.pop()
        stack.push(f"({op1} {char} {op2})")

return stack.pop()
```

Prefix to postfix changes the push to `f"{op1} {op2} {char}"`.

Infix to postfix — `Infix_to_postfix.py`

```
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}

output = ""
operator_stack = Stack()
i = 0

while i < len(expression):
    char = expression[i]

    if char.isdigit():
        number = ""
        while i < len(expression) and expression[i].isdigit():
            number += expression[i]
            i += 1
        output += number + " "
        i -= 1

    elif char == "(":
        operator_stack.push(char)

    elif char == ")":
        while not operator_stack.is_empty() and operator_stack.peek() != "(":
            output += operator_stack.pop() + " "
        operator_stack.pop()

    elif char in PRECEDENCE:
        while not operator_stack.is_empty() and operator_stack.peek() != "(" and
PRECEDENCE[operator_stack.peek()] >= PRECEDENCE[char]:
            output += operator_stack.pop() + " "
```

```
            operator_stack.push(char)

        elif not char.isspace():
            output += char + " "

        i += 1

    while not operator_stack.is_empty():
        output += operator_stack.pop() + " "

    return output.strip()
```

Infix to prefix — `infix_to_prefix.py`

```
    reversed_exp = ""

    for char in reversed(expression):

        if char == "(":
            reversed_exp += ")"

        elif char == ")":
            reversed_exp += "("

        else:
            reversed_exp += char
```

Then the infix-to-postfix loop on `reversed_exp`. `prefix = output[::-1]`.